



Structure (DDL)

Create & Drop Objects

```
-- Create Table with Constraints
CREATE TABLE Employees (
    Id INT PRIMARY KEY IDENTITY(1,1), -- Auto Inc
    Name VARCHAR(50) NOT NULL,       -- Required
    Email VARCHAR(100) UNIQUE,        -- No Duplicates
    PhoneNumber VARCHAR(50),
    DeptId INT REFERENCES Depts(Id),  -- Foreign Key
    Age INT CHECK (Age >= 18)         -- Validation
);

-- Drop Table (Structure + Data)
DROP TABLE IF EXISTS Employees;
```

Alter Table (Modify Structure)

```
-- Add a new column
ALTER TABLE Employees ADD Status VARCHAR(10);

-- Drop a column
ALTER TABLE Employees DROP COLUMN Status;

-- Modify Column Data Type
ALTER TABLE Employees ALTER COLUMN Age FLOAT;
```

Constraints & Renaming

```
-- Add Constraint:
ALTER TABLE Employees
ADD CONSTRAINT UK_PhNum UNIQUE(PhoneNumber);

-- Drop Constraint:
ALTER TABLE Employees
DROP CONSTRAINT UK_PhNum;

-- Rename Column: 'Table.Old', 'New', 'COLUMN'
EXEC sp_rename 'Users.Surname', 'LastName', 'COLUMN';

-- Rename Table: 'Old', 'New'
EXEC sp_rename 'OldTable', 'NewTable';
```

Common Data Types

Text Strings

- VARCHAR(n)** : 8-bit ASCII (1 byte/char).
- NVARCHAR(n)** : Unicode (2 bytes, for Arabic/Emoji).
- CHAR(n)** : Fixed length (pads with spaces).

Numeric

- INT** : Whole number (± 2 billion).
- BIGINT** : Large integer (Financial/Big Data).
- DECIMAL(p,s)** : Exact precision (e.g., Currency).
- BIT** : 0, 1, or NULL (Used as Boolean).

Date & Time

- DATETIME2** : Modern standard (High precision).
- DATE** : Date only (YYYY-MM-DD).
- TIME** : Time only (HH:MM:SS).

Joins (Relationships)

Table 1 (Left)	Table 2 (Right)
A	A
B	B
C	D

INNER JOIN

Matches in both tables.

```
SELECT * FROM t1 INNER JOIN t2 ON t1.id = t2.id;
```

LEFT JOIN

All Left + Matching Right.

```
SELECT * FROM t1 LEFT JOIN t2 ON t1.id = t2.id;
```

RIGHT JOIN

All Right + Matching Left.

```
SELECT * FROM t1 RIGHT JOIN t2 ON t1.id = t2.id;
```

FULL JOIN

All records from both.

```
SELECT * FROM t1 FULL JOIN t2 ON t1.id = t2.id;
```

CROSS JOIN (Cartesian Product)

Every row in T1 paired with every row in T2.

```
SELECT * FROM t1 CROSS JOIN t2; -- No ON clause
```

Set Operators

Combine result sets. Columns/Types must match.

UNION

Unique rows from both (Deduped).

```
SELECT c FROM t1 UNION SELECT c FROM t2;
```

UNION ALL

All rows, including duplicates.

```
SELECT c FROM t1 UNION ALL SELECT c FROM t2;
```

INTERSECT

Rows common to both sets.

```
SELECT c FROM t1 INTERSECT SELECT c FROM t2;
```

EXCEPT

In First, not in Second.

```
SELECT c FROM t1 EXCEPT SELECT c FROM t2;
```

Analysis & Aggregation

Aggregate Functions

- COUNT(*)** : Counts all rows (including NULLs).
- COUNT(col)** : Counts only non-NULL values.
- SUM(col)** : Calculates the total sum of the column.
- AVG(col)** : Calculates the average value.
- MIN(col)** : Finds the smallest value.
- MAX(col)** : Finds the largest value.
- STRING_AGG(col, ' ', '')** : Joins values into one string.

Grouping Strategy

```
SELECT Dept, YEAR(HireDate), COUNT(*)
FROM Employees
WHERE Salary > 50000 -- 1. Filter Rows
GROUP BY Dept, YEAR(HireDate) -- 2. Create Buckets
HAVING COUNT(*) > 5; -- 3. Filter Buckets
```

CASE Statement (Logic)

```
SELECT Name,
CASE
    WHEN Age < 18 THEN 'Minor'
    WHEN Age >= 65 THEN 'Senior'
    ELSE 'Adult'
END AS Category
FROM Users;
```

Querying Operators

Comparison

=	Equal
<> / !=	Not Equal
> / >=	Greater / Greater or Equal
< / <=	Less / Less or Equal
IS NULL	Missing value
IS NOT NULL	Not Missing value

Logical

AND	Both True
OR	Either True
NOT	Invert Result
BETWEEN	Range (Inclusive)
IN	List Match

 Never use = NULL. Always use IS NULL.

Wildcards (used with LIKE)

'%A'	Ends with A (0+ chars before)
'A%'	Starts with A (0+ chars after)
'_A'	Exactly 2 chars, ends in A
'[AB]'	Starts with A OR B
'[^C]'	Does NOT start with C

Sorting & Paging

```
SELECT TOP(10) * -- Standard Limit
FROM Sales
ORDER BY Amount DESC -- Sort First
OFFSET 10 ROWS -- Skip first 10
FETCH NEXT 5 ROWS ONLY; -- Take next 5
```



Modifying Data (DML)

Insert Data

```
-- 1. Insert Single Row
INSERT INTO Users (Name, Age) VALUES ('Alice', 30);

-- 2. Insert Multiple Rows (Bulk)
INSERT INTO Users (Name, Age) VALUES
('Bob', 25),
('Charlie', 35),
('David', 40);

-- 3. Insert from another table
INSERT INTO Archive (Name, Age)
SELECT Name, Age FROM Users WHERE Active = 0;

-- 4. Create & Fill New Table (Temp Table)
SELECT * INTO #TempUsers FROM Users;
```

Update Data

```
-- Standard Update
UPDATE Products
SET Price = Price * 1.1
WHERE CategoryId = 5;

-- Update using Join
UPDATE t
SET t.Price = s.NewPrice
FROM Products t
JOIN SupplierUpdates s ON t.Id = s.Id;
```

DML with Output Return data from modified rows instantly.

```
-- Return generated ID after Insert
INSERT INTO Users (Name)
OUTPUT inserted.Id
VALUES ('Eve');

-- Capture deleted data (e.g. for archiving)
DELETE FROM Users
OUTPUT deleted.Id, deleted.Name
WHERE Active = 0;
```

Deleting Data

- **DELETE FROM t WHERE c=1**: Removes specific rows (Logged, Slower).

 Always use WHERE, or you will wipe the whole table!

- **TRUNCATE TABLE t**: Removes ALL rows (Minimal Log, Instant).

Indexing

```
-- Indexing: Speeds up reads, slows down writes
CREATE INDEX idx_name ON Users(LastName);
DROP INDEX idx_name ON Users;
```

Scalar Functions

Functions

- **Null Handling**
 - **ISNULL(col, val)**: Uses val if column is NULL.
 - **COALESCE(v1, v2, ...)**: Returns the first non-NULL value.
 - **NULLIF(v1, v2)**: Returns NULL if v1 = v2.
- **String Parsing**
 - **LEN(col)**: Returns the length of a string.
 - **REPLACE(col, 'old', 'new')**: Replaces 'old' occurrences with 'new'.
 - **CHARINDEX('txt', col)**: Returns the starting position of text.
 - **SUBSTRING(col, start, len)**: Extracts text part.
 - **UPPER(col)**: Converts text to Upper case.
 - **LOWER(col)**: Converts text to Lower case.
 - **LEFT(col, 3)**: Extracts first 3 characters.
 - **RIGHT(col, 3)**: Extracts last 3 characters.
 - **TRIM(col)**: Removes leading/trailing spaces.
 - **CONCAT(str1, str2)**: Joins text safely (treats NULL as empty).
- **Date Math**
 - **GETDATE()**: Returns current System DateTime.
 - **DATEADD(DAY, 7, Col)**: Adds 7 days to date.
 - **DATEADD(DAY, -7, Col)**: Subtracts 7 days from date.
 - **DATEDIFF(DAY, A, B)**: Counts days between two dates.
 - **EOMONTH(Date)**: Returns last day of month.
- **Conversion**
 - **CAST(val AS INT)**: Converts type (Error on fail).
 - **TRY_CAST(val AS INT)**: Returns NULL on fail.
 - **FORMAT(dt, 'yyyy-MM-dd')**: Custom formatting.

Logical Order of Execution

Logical Order of Execution How SQL actually processes your query (not how you write it).

- 1 FROM, JOIN, CTEs**
Gather & Link Datasets
- 2 WHERE**
Filter Rows (Pre-Aggregation)
- 3 GROUP BY**
Bucket Data into Groups
- 4 HAVING**
Filter Groups (Post-Aggregation)
- 5 SELECT**
Window Funcs & Expressions
- 6 SET OPERATIONS**
UNION, INTERSECT, EXCEPT
- 7 ORDER BY**
Sort the Final Results
- 8 TOP & OFFSET**
Limit Displayed Rows

Subqueries, CTEs, Window Functions

Single Row Subquery Returns exactly one value (scalar).

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Multi-Row Subquery (IN) Returns a list of values.

```
SELECT name
FROM products
WHERE category_id IN (
SELECT id FROM categories WHERE name = 'Tech'
);
```

Correlated Subquery Inner query depends on Outer query (Runs for each row).

```
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (
SELECT AVG(salary)
FROM employees
WHERE dept_id = e.dept_id -- Link to outer
);
```

EXISTS Operator True if subquery returns ANY rows. High performance.

```
SELECT name
FROM customers c
WHERE EXISTS (
SELECT 1 FROM orders o WHERE o.cust_id = c.id
);
```

CTEs (Common Table Expressions) A readable, modular alternative to subqueries.

```
WITH HighSales AS (
-- Define the temporary result set
SELECT Region, SUM(Amount) as Total
FROM Sales GROUP BY Region
)
SELECT * FROM HighSales WHERE Total > 10000;
```

Window Functions Performs calculations across a set of rows related to the current row.

```
SELECT Dept, Name, Salary,
-- Rank employees by salary within their Dept
RANK() OVER (
PARTITION BY Dept
ORDER BY Salary DESC
) as Rnk,
-- Running Total of salary
SUM(Salary) OVER (ORDER BY HireDate) as RunTotal
FROM Employees;
```

- **ROW_NUMBER()**: Unique sequential ID (1, 2, 3, 4).
- **RANK()**: Skips on ties (1, 1, 3, 4).
- **DENSE_RANK()**: No skips on ties (1, 1, 2, 3).
- **NTILE(4)**: Splits rows into 4 buckets.
- **LEAD(col, 1)**: Value from NEXT row.
- **LAG(col, 1)**: Value from PREVIOUS row.